SHGNet-56

Ear Landmark & Piercing Detection Pipeline

Technical documentation: how data, annotation, training, augmentation, export, and live inference fit together.

Repository: github.com/santoshreddy-hash/2shgnet-try-on

Project folder: shgnet56_train

1. Overview

SHGNet-56 extends a stacked hourglass ear-landmark network from 55 anatomical ear points to 56 points by adding a piercing landmark (#56). The system supports annotation, multi-stage training, offline data augmentation, ONNX export, and real-time inference on desktop or in the browser.

Goals

- Localize 55 ear landmarks + 1 piercing point on a cropped ear image.
- Train from a pretrained 55-landmark checkpoint with progressive unfreezing.
- Run live try-on / overlay using YOLO ear tip + SHGNet-56 heatmaps.

Core I/O

- Model input: 256x256 RGB ear crop
- Model output: 56 Gaussian heatmaps (64x64), decoded to pixel landmarks
- Piercing index: landmark #56 (0-based index 55)

2. Repository Layout

```
shgnet56_train/
  annotator/     Gradio one-click piercing annotator
  train/         Dataset, augment, model, train, eval, ONNX, aug export
  live/          Desktop live ONNX inference
  tracking/      One-Euro smoothing helpers
  web/           Browser demo (onnxruntime-web)
  data/          Datasets (local; not in git)
  models/        Weights stubs / local .pth .onnx
  outputs/       Checkpoints, aug images, logs
  run_pipeline.py  annotate | train | export
```

3. End-to-End Pipeline

High-level flow from raw images to live piercing prediction:

```
1. Collect ear images (ear_pose YOLO layout and/or iBUG crops)
2. Annotate piercing #56  -->  labels/*.txt or sibling .pts
3. Train SHGNet-56 (Stage1 -> Stage2 -> Stage3)
4. Optional: export additive augmentations for documentation / extra data
5. Export ONNX (SHGNet-56.onnx)
6. Live: YOLO tip crop -> SHGNet-56 -> remap to frame -> One Euro smooth
```

CLI entry

```
python run_pipeline.py annotate
python run_pipeline.py train
python run_pipeline.py export
python run_pipeline.py all          # train then export
```

4. Data Formats

4.1 YOLO ear_pose (primary)

Layout mirrors Ultralytics YOLO-Pose:
  images/{train,val}/*.png
  labels/{train,val}/*.txt

Each label line: class cx cy w h + (x y v) * N keypoints, normalized 0-1.
For piercing training, N must be 56 (visibility v>0 for #56).

4.2 iBUG-style .pts

Used for collectiona crops: image + sibling .pts with n_points and x y pairs. Piercing is the 56th point when present.

4.3 Datasets used in this project

- ear_pose train: 1,700 images (annotate #56 in Documents labels/train)
- ear_pose val: 300 images (with #56)
- collectiona train/test: 500 + 105 crops (optional)

Important: training only uses samples that already contain piercing #56. If train labels have only 55 points, those images are skipped for SHGNet-56.

5. Annotation

Gradio app (annotator/app.py): open an image folder, click the piercing location, save into the matching YOLO .txt or .pts file as landmark #56.

```
python annotator/app.py --folder data/data/ear_pose/images/val --port 7860
```

- YOLO mode writes/overwrites kpt #56 in labels/<split>/<stem>.txt
- PTS mode writes landmark #56 into sibling .pts

6. Training (3 Stages)

Training starts from a pretrained 55-LM hourglass and adapts to 56 outputs. Each stage unfreezes more weights and uses a lower learning rate.

Stage 1 - Output heads only

- Trainable: heatmap output layers
- Default: 30 epochs, LR 1e-3
- Purpose: quickly learn piercing channel while backbone frozen

Stage 2 - Last hourglass

- Trainable: last hourglass stack + features + outputs
- Default: 20 epochs, LR 1e-4
- Purpose: adapt ear features for 56 landmarks

Stage 3 - Full network

- Trainable: all parameters
- Default: 15 epochs, LR 1e-5
- Purpose: low-LR polish; best weights become SHGNet-56_final.pth

Online augmentation (during train)

Applied on-the-fly to training batches (not validation): horizontal flip, rotation +/-15 deg, scale 0.9-1.1, translate +/-5%, brightness/contrast +/-0.2, Gaussian blur. Landmarks are transformed with geometric augs.

Metrics

- NME: mean landmark error / crop diagonal (sqrt(2)*256)
- PCK@0.02 / 0.05 / 0.10

- Piercing error in pixels and normalized units

Example fine-tune command

```
python -m train.train \
  --images-dir data/data/ear_pose/images/train \
  --from-stage2 \
  --checkpoint outputs/checkpoints/SHGNet-56_final.pth \
  --stage2-epochs 12 --stage3-epochs 6 \
  --device cuda   # or mps / cpu
```

7. Offline Additive Augmentation

Script: train/export_augmented_dataset.py
For documentation / expanded datasets, each source image produces 44 variants (additive families, not a full cartesian product).

```
flip 2 + scale 5 + translate 7 + blur 5 + noise 6
+ occlusion 6 + smoke_blur 5 + brightness 4 + contrast 4  =  44
```

- Geometric augs move landmarks; photometric augs keep coords.
- Outputs under outputs/augmented/ and flattened outputs/augmented_all/
- images/ and coords/ (or labels/) kept separate after organization

If source train labels lacked #56, aug labels also lacked #56. Annotated labels from labels.zip were later used to regenerate 74,800 train-aug YOLO labels with piercing.

8. ONNX Export

```
python -m train.export_onnx --checkpoint outputs/checkpoints/SHGNet-56_final.pth
```

Produces SHGNet-56.onnx for desktop (onnxruntime) and browser (onnxruntime-web). YOLO pose ONNX is a separate detector used only to find the ear tip / crop.

9. Live Inference

Pipeline per frame

```
Camera frame
  -> YOLO pose: detect person, pick ear tip
  -> Square tip-centered ear crop (+ gray pad), LEFT ear flipped
  -> Resize 256x256 -> SHGNet-56 -> 56 points in crop space
  -> Remap to full frame (undo flip/scale/origin)
  -> Tip-relative hold + One Euro filter for stable overlay
```

Desktop

```
python -m live.desktop_onnx
```

Browser

```
cd web && npm install && npm start   # serve at http://127.0.0.1:8765
```

Live alignment depends on side-profile visibility and jewellery-style tip-hold: landmarks are stored relative to the YOLO tip and re-anchored when the tip updates.

10. Size & Runtime Notes

- Input 256, heatmap 64, crop pad ~1.65 x pinna height

- Batch size default 8 (raise on GPU if VRAM allows)
- Mac MPS: ~minutes per epoch on ~1.4k images; days if training all 114k aug files
- Recommended: train on annotated originals with online aug; use offline aug selectively

11. Google Colab (remote GPU)

```
!git clone https://github.com/santoshreddy-hash/2shgnet-try-on.git
%cd 2shgnet-try-on
!pip install torch torchvision opencv-python-headless numpy Pillow tqdm
# Mount Drive, place images/labels + SHGNet-56_final.pth
!python -m train.train --images-dir data/data/ear_pose/images/train \
    --from-stage2 --checkpoint outputs/checkpoints/SHGNet-56_final.pth \
    --device cuda --batch-size 16
```

Use Drive for dataset and checkpoints so free Colab disconnects do not lose progress. Pass --images-dir to avoid Mac absolute paths in train/config.py.

12. Component Summary

```
Annotator  -->  Labels (#56)
     |
     v
Dataset loader (+ online aug)  -->  3-stage Trainer  -->  .pth
     |                                      |
     +--> Offline aug export (optional)     +--> ONNX export
                                              |
                                YOLO crop + Live / Web overlay
```

13. Key Paths (local)

```
Code: .../56 shgnet extended/shgnet56_train
Annotated labels: .../Documents/.../ear_pose/labels/train
Aug images: outputs/augmented_all/images
Aug coords: outputs/augmented_all/coords
Train-aug labels zip: outputs/ear_pose_train_aug_labels_56.zip
Checkpoints: outputs/checkpoints/
GitHub: https://github.com/santoshreddy-hash/2shgnet-try-on
```

Document generated for the SHGNet-56 / 2shgnet-try-on project. Epoch counts and paths may be overridden by CLI flags.